@ 2025

:: OWASP Top 10 2025: IAAA Failures

Introduction

This room provides a beginner-friendly breakdown of **three categories from the OWASP Top 10 (2025)**. It focuses on security failures related to how **Identity, Authentication, Authorization, and Accountability (IAAA)** are implemented within web applications. These areas are critical because weaknesses in them can allow attackers to gain unauthorized access, bypass login systems, or perform malicious actions without detection.

Through this room, the learner will first understand the core theory behind each vulnerability category and then apply that knowledge by solving practical supporting challenges. This hands-on approach helps bridge the gap between conceptual understanding and real-world exploitation scenarios.

The following OWASP Top 10 categories are covered:

- **A01: Broken Access Control**
- **A07: Authentication Failures**
- **A09: Logging & Monitoring Failures**

What is IAAA?

IAAA is a simple way to understand how applications verify users and track their actions. It has **four important parts**, and each part depends on the previous one. This means you **cannot skip any step** because every later step works only if the earlier one is done correctly.

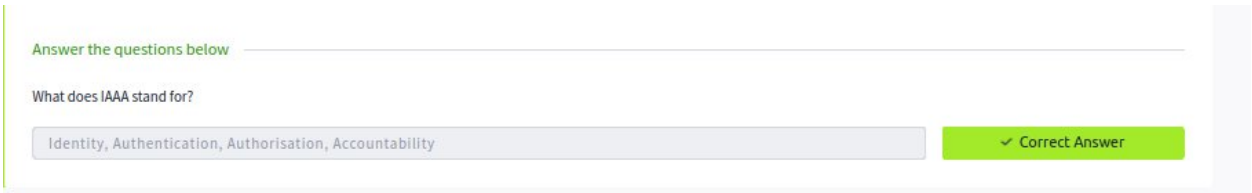
The four parts are:

- **Identity** – The unique account that represents a user or service, such as a **user ID or email address**.
- **Authentication** – The process of proving that the user really owns that identity, for example by using a **password, OTP, or passkey**.

- **Authorization** – Defines **what actions or resources the authenticated user is allowed to access**.
- **Accountability** – Keeps a record of **who performed an action, when it happened, and from where**, along with alerts for suspicious activity.

The **OWASP Top 10:2025 categories** covered in this room are mainly related to failures in how **IAAA is implemented**. If these controls are weak, attackers may be able to **access other users' data, bypass restrictions, or gain higher privileges than they should have**.

Understanding IAAA is important because it forms the **foundation of application security**. Learning these concepts helps in identifying common security issues such as broken access control, weak authentication, and missing logging mechanisms.



A01 :: Broken Access Control

Broken Access Control happens when a web application **fails to properly check whether a user is allowed to access a specific resource or perform a certain action**.

In simple words, the server should verify **"Is this user really allowed to see this data?"** every time a request is made.
If the server does not check this properly, users may be able to **view, edit, or delete data that belongs to someone else**.

A very common example of this is called **IDOR (Insecure Direct Object Reference)**.

:: What is IDOR?

IDOR happens when the application directly uses user-controlled input, such as an **account ID in the URL**, without checking whether the logged-in user actually owns that account.

For example:

```
https://example.com/account?id=7
```

Here, the user is viewing account **ID 7**.

If the user manually changes the URL to:

```
https://example.com/account?id=6
```

and the application shows another user's account details, then the application has **Broken Access Control**.

This is dangerous because the application is **trusting the client too much** instead of verifying permissions on the server side.

:: Types of Privilege Escalation

Broken Access Control often leads to **privilege escalation**, which means gaining access beyond what a normal user should have.

1 :: Horizontal Privilege Escalation

This happens when a user accesses **another user's data with the same privilege level**.

Example:

- User A logs in
- Changes `accountID=101` to `accountID=102`
- Now sees User B's bank details

Both are normal users, but one accessed another's data.

2 :: Vertical Privilege Escalation

This happens when a normal user gains access to **admin-level features**.

Example:

- A user changes `/profile` to `/admin`
- The server allows access without checking role
- User gets admin dashboard

This is more dangerous because it can lead to **full control of the application**.

Practical Task Explanation

In this task, you need to start the provided **static site** and observe the **`accountID` value in the URL**.

The main goal is to:

1. Open the account page
2. Look at the URL parameter like `?accountID=1`
3. Change the value manually
4. Check different users' account balances
5. Find the user whose balance is **more than $1 million**

This helps you understand how simply changing a number in the URL can expose sensitive data if access control is missing.

Why This is Dangerous

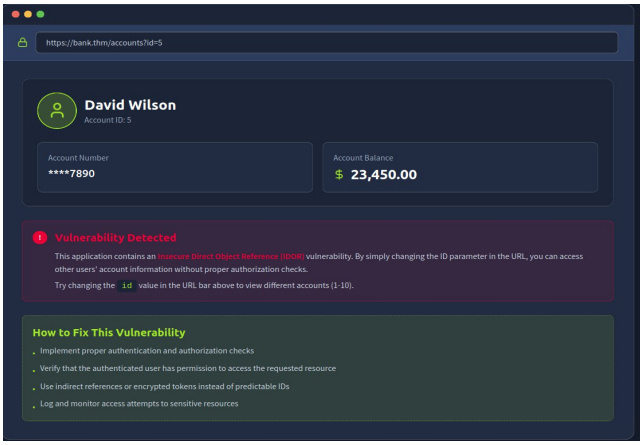
If this vulnerability exists in real applications, attackers can:

- View private user information
- Access bank accounts
- Change other users' data
- Delete records
- Gain admin access

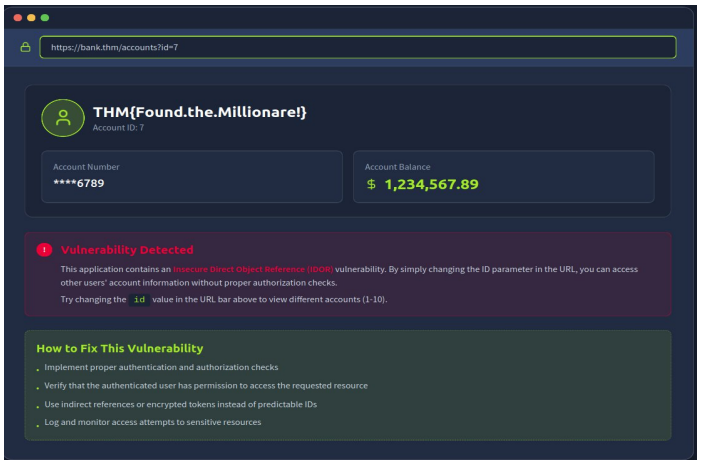
That is why **server-side authorization checks are mandatory for every request**.

Lab:

Click view site,



Task : What is the note you found when viewing the user's account who had more than $ 1 million?



Change the object reference,

../accounts?id=5

Then change the references,

../accounts?id=7

A07 :: Authentication Failures

Authentication Failures occur when a web application **cannot properly verify that a user is really who they claim to be**.

In simple terms, authentication is the **login verification process**.
Whenever a user enters a username and password, the application must correctly confirm the identity before giving access.

If this process is weak or has logical mistakes, attackers may be able to **log in as another user, hijack sessions, or even access admin accounts**.

Common Causes of Authentication Failures

Some common reasons include:

- **Username Enumeration** – The application reveals whether a username exists or not.
- **Weak or Guessable Passwords** – Simple passwords that attackers can easily guess.
- **No Lockout / No Rate Limiting** – Unlimited login attempts allow brute-force attacks.
- **Logic Flaws in Login or Registration** – Mistakes in how usernames are checked or accounts are created.
- **Insecure Session or Cookie Handling** – Session IDs can be stolen, reused, or manipulated.

If any of these weaknesses exist, attackers may **impersonate other users or bind their session to the wrong account**.

Practical Task Explanation

In this lab, the target username is already known: **`admin`**.

The challenge is to exploit a **logic flaw in the registration system**.

The trick is to register a new account using the username:

`aDmiN`

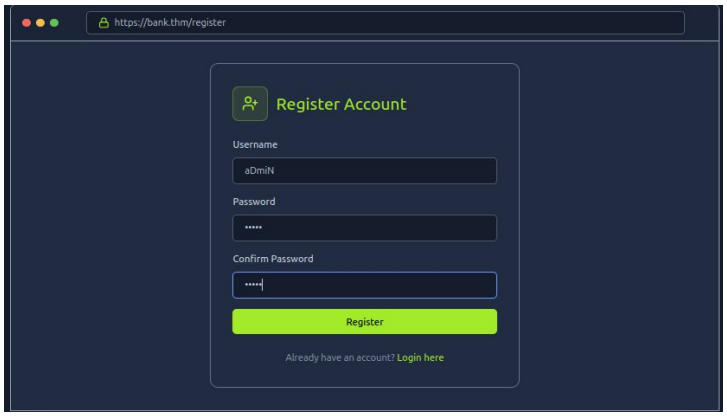
Even though it looks different, some poorly designed applications treat usernames as **case-insensitive** during login but **case-sensitive during registration**.

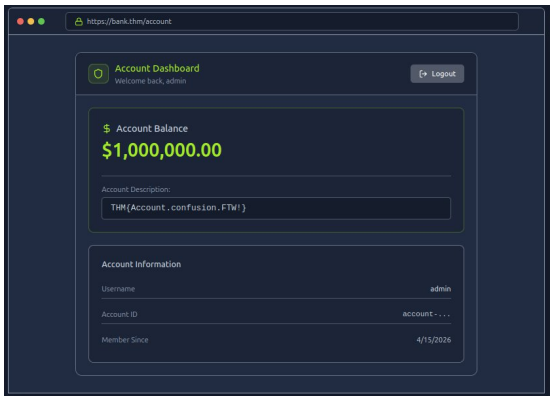
This means:

- The system may allow registration of **`aDmiN`**
- During login, it may normalize it to **`admin`**
- The session may get linked to the real **admin account**

As a result, the attacker may successfully log in as the **administrator**.

This is a classic example of an **authentication logic flaw**.

Snap1 : 

Snap2: 

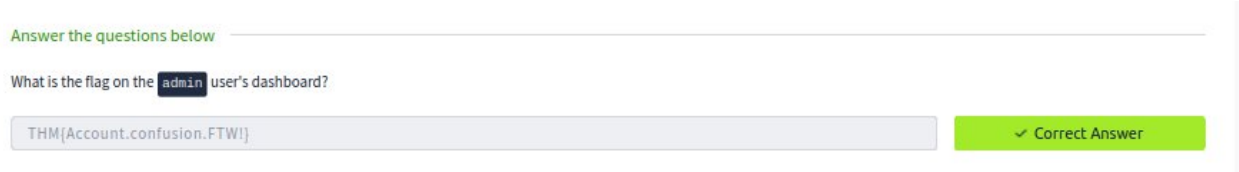
Why This Happens

The problem occurs because the application is **not consistently validating usernames**.

For example:

- Registration treats `admin` and `aDmiN` as different
- Login treats them as the same
- Session gets attached to the wrong identity

This mismatch allows attackers to bypass normal authentication rules.



Real-World Impact

If authentication failures exist in real systems, attackers can:

- Log in as other users
- Access admin dashboards
- Reset passwords
- Take over accounts
- Bypass MFA workflows
- Steal sessions using weak cookies

This lab teaches that **authentication is not just about passwords**.
Even small mistakes in **username handling, case sensitivity, or session logic** can lead to **complete account takeover**.

That is why developers must:

- Normalize usernames consistently
- Enforce strong passwords
- Add account lockout
- Use secure cookies
- Validate sessions properly

A09 :: Logging & Monitoring Failures

Logging and Monitoring Failures happen when an application **does not properly record important security events or fails to alert defenders when suspicious activity occurs**.

In simple terms, whenever users log in, change passwords, access sensitive data, or attempt suspicious actions, the application should **store these events in logs**. These logs help security teams understand **who performed an action, when it happened, and from where it was done**.

This is an important part of **accountability**, because without logs there is no proof of what happened during an attack.

Why Logging is Important

Good logging helps defenders to:

- Detect attacks quickly
- Investigate suspicious activity
- Identify affected accounts
- Trace attacker actions
- Find the source IP address
- Understand the attack timeline
- Collect evidence for incident response

Without proper logs, attackers may remain hidden for a long time.

Common Logging Failures

Some common examples include:

- **Missing authentication logs**
  No records of successful or failed login attempts.
- **Vague or unclear error logs**
  Logs do not clearly mention what action failed.
- **No alerting system**
  No warning when brute-force attacks or privilege changes happen.
- **Short log retention**
  Logs are deleted too quickly, making investigations difficult.
- **Logs stored insecurely**
  Attackers may delete or modify the logs to hide evidence.

These failures make it hard to understand what happened during a security incident.

Practical Task Explanation

In this lab, you need to investigate an application that is under attack.

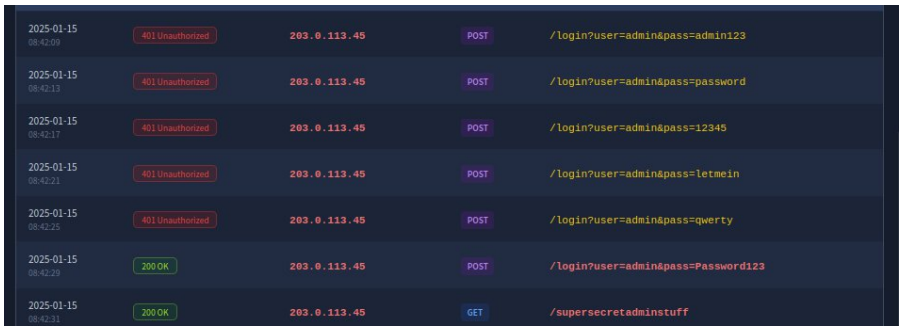
The task helps you understand **what information is required during a real incident investigation**.

You need to:

1. Start the static site
2. Review the provided logs
3. Identify suspicious actions
4. Track what the attacker did
5. Answer the questions based on the evidence

Question1:

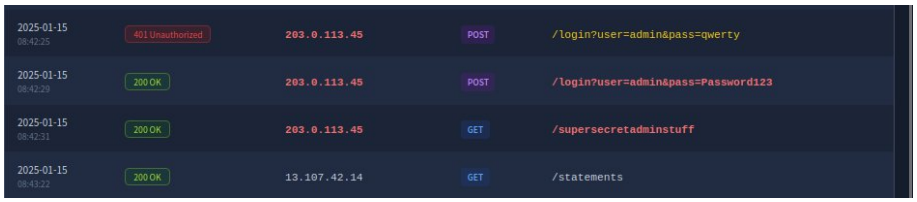
Proof:



2025-01-15 08:42:09	401 Unauthorized	203.0.113.45	POST	/login?user=admin&pass=admin123
2025-01-15 08:42:13	401 Unauthorized	203.0.113.45	POST	/login?user=admin&pass=password
2025-01-15 08:42:17	401 Unauthorized	203.0.113.45	POST	/login?user=admin&pass=12345
2025-01-15 08:42:21	401 Unauthorized	203.0.113.45	POST	/login?user=admin&pass=letmein
2025-01-15 08:42:25	401 Unauthorized	203.0.113.45	POST	/login?user=admin&pass=qwerty
2025-01-15 08:42:29	200 OK	203.0.113.45	POST	/login?user=admin&pass=Password123
2025-01-15 08:42:31	200 OK	203.0.113.45	GET	/supersecretadminstuff

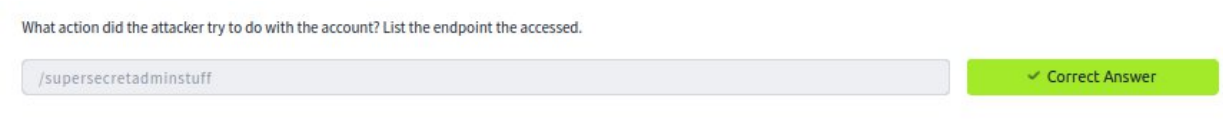
Question2:

Proof:

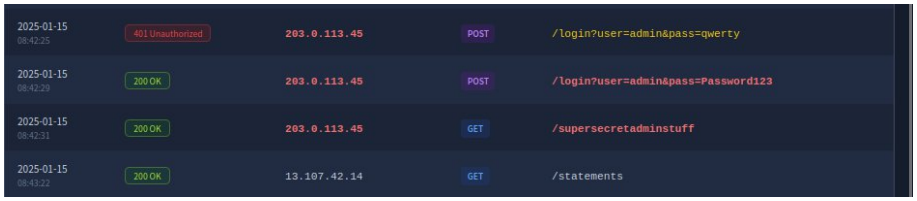


2025-01-15 08:42:25	401 Unauthorized	203.0.113.45	POST	/login?user=admin&pass=qwerty
2025-01-15 08:42:29	200 OK	203.0.113.45	POST	/login?user=admin&pass=Password123
2025-01-15 08:42:31	200 OK	203.0.113.45	GET	/supersecretadminstuff
2025-01-15 08:43:22	200 OK	13.107.42.14	GET	/statements

Question3:



Proof:



2025-01-15 08:42:25	401 Unauthorized	203.0.113.45	POST	/login?user=admin&pass=qwerty
2025-01-15 08:42:29	200 OK	203.0.113.45	POST	/login?user=admin&pass=Password123
2025-01-15 08:42:31	200 OK	203.0.113.45	GET	/supersecretadminstuff
2025-01-15 08:43:22	200 OK	13.107.42.14	GET	/statements

While doing this, think about how difficult the investigation would become if important details were missing, such as:

- login timestamps
- username used
- source IP address
- failed login attempts
- privilege changes
- accessed resources

Even one missing detail can make it much harder to reconstruct the attack.

Real-World Impact

If applications lack proper logging and monitoring:

- attacks may go unnoticed
- stolen accounts cannot be traced
- insider threats are harder to detect
- brute-force attacks may continue silently
- evidence may be lost
- incident response becomes delayed

This can significantly increase the damage caused by an attacker.

This lab teaches that **security is not only about preventing attacks, but also about detecting and investigating them**.

Even if an attacker gets inside, good logs allow defenders to:

- understand the incident
- respond quickly

- reduce damage
- improve future defenses

Without proper logging, organizations may never know **how the attacker entered, what they changed, or what data was stolen**.